

VirtuEx 시스템 감사 보고서 (20차)

VirtuEx System Audit Report (20th)

- 감사일: 2026-03-19
- 감사 범위: API 보안, 입력 검증, 에러 처리, 인증/인가, DB 무결성, 서비스 간 통신, 로깅/모니터링, 프론트엔드 보안, 데이터 노출
- 감사 방법: 전체 소스 코드 정적 분석 + API 흐름 추적 + 보안 취약점 패턴 매칭
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 본 보고서는 발견 사항만 기록하며, 수정은 포함하지 않음

이전 감사 대비 수정 현황 (19차 → 20차)

이전 이슈	상태	비고
[SEC-C-01] JWT 액세스 토큰 sessionStorage 저장	미수정	여전히 sessionStorage에 저장, XSS 취약점 유지 (아키텍처 변경 필요)
[AUTH-M-01] 관리자 체결 감사 엔드포인트 권한 미검증	수정됨	@UseGuards(JwtAuthGuard, AdminRolesGuard) 추가 확인
[AUTH-M-02] 거래 통계 엔드포인트 권한 미검증	수정됨	@UseGuards(JwtAuthGuard, AdminRolesGuard) 추가 확인
[VAL-M-01] 주문 수정 body DTO 미검증	수정됨	@Body() body: ModifyOrderDto 타입 적용 확인
[GW-M-01] JwtStrategy localhost 하드코딩	미수정	배포 환경 장애 가능성 유지
[CSP-M-01] Content Security Policy unsafe-inline	부분 수정	프로덕션에서 'strict-dynamic' 추가됨, 그러나 'unsafe-inline'도 동시 포함 (하단 참조)

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 인증/토큰 보안 (미수정)	1	0	0	0	1
B. API 입력 검증	0	0	2	1	3
C. 에러 처리/정보 노출	0	0	2	1	3
D. XSS/CSP 방어	0	0	2	0	2
E. 인가/권한 검증	0	0	1	1	2
F. 서비스 간 통신	0	0	1	1	2
G. 프론트엔드 보안	0	0	1	1	2
H. 데이터 무결성	0	0	1	1	2

합계	1	0	10	6	17
----	---	---	----	---	----

A. 인증/토큰 보안 (미수정) — 1건

[SEC-C-01] JWT 액세스 토큰 sessionStorage 저장 — XSS 취약점 (Critical, 미수정)

현상: JWT 액세스 토큰이 sessionStorage에 저장되어 페이지 내 실행되는 모든 JavaScript에서 sessionStorage.getItem('accessToken')으로 접근 가능. auth.ts 상단에 상세한 4단계 마이그레이션 계획 (HttpOnly 쿠키 전환, CSRF 보호, WebSocket 인증 등)이 명시되어 있으나 미수행 **위치:** frontend/src/stores/auth.ts:89-119 **권장:** 마이그레이션 계획(Step 1~4)에 따라 단계적으로 HttpOnly 쿠키 기반 인증으로 전환. 프로덕션 출시 전 최우선 수행 **심각도:** Critical

B. API 입력 검증 — 3건

[VAL-M-01] 채팅 메시지 — content 길이 제한 미검증 (Medium)

현상: 채팅 메시지 전송 시 프론트엔드 측 길이 제한이 없음. 백엔드 SendMessageDto에서 @MaxLength() 데코레이터 적용 여부 미확인. 극단적으로 긴 메시지(수 MB) 전송 시 DB 저장 및 다른 사용자의 렌더링 성능 저하 가능 **위치:** frontend/src/hooks/useChat.ts (sendMessage), backend/services/chat/src/chat/dto/send-message.dto.ts **영향:** 대량 메시지로 인한 DB 부하 및 채팅 UI 렌더링 성능 저하 **권장:** 프론트엔드 MessageInput에서 maxLength 제한 추가 (5000자), 백엔드 DTO에 @MaxLength(5000) 확인 **심각도:** Medium

[VAL-M-02] 주문 수량 — 프론트엔드 소수점 자릿수 미제한 (Medium)

현상: OrderForm.tsx에서 수량 입력 시 사용자가 직접 입력하면 소수점 20자리 이상 입력 가능. 비율 버튼 사용 시 .toFixed(8)로 포맷되지만 직접 입력 시에는 제한 없음. 백엔드에서 Decimal 처리 시 정밀도 손실 또는 처리 비용 증가 **위치:** frontend/src/components/trading/OrderForm.tsx:304 **영향:** 극단적 소수점 값으로 인한 백엔드 연산 비용 증가 **권장:** 입력 필드에서 소수점 8자리 제한 또는 onBlur 시 .toFixed(8) 적용 **심각도:** Medium

[VAL-L-01] 입금/출금 금액 — 음수 입력 방지 미흡 (Low)

현상: portfolio/page.tsx:80-82, 104-106의 입금/출금 핸들러에서 amount <= 0 체크가 있으나, HTML type="number" 입력에서 음수 값 입력이 가능하며 parseFloat로 변환 시 -100 등이 유효한 숫자로 통과. e.target.value에 음수가 입력되면 상태에 저장됨 **위치:** frontend/src/app/(main)/portfolio/page.tsx:81, 105 **영향:** 음수 금액이 잠깐 UI에 표시될 수 있음 (서버에서 거부되지만 UX 혼란) **권장:** Input에 min="0" 속성 추가 또는 onChange에서 음수 입력 차단 **심각도:** Low

C. 에러 처리/정보 노출 — 3건

[ERR-M-01] 주문 프록시 — 거래 알림 실패 시 내부 에러 메시지 로깅 (Medium)

현상: order-proxy.controller.ts:62에서 sendTradeNotification 실패 시 e.message를 Logger.warn으로 출력. 프로덕션 환경에서 내부 서비스 URL, 인증 토큰 관련 에러가 로그에 노출될 가능성 **위치:** backend/services/api-gateway/src/proxy/order-proxy.controller.ts:62 **영향:** 로그 수집 시스템에서 민감 정보 노출 가능 **권장:** 에러 메시지를 일반화하거나 e.message 대신 에러 코드만 기록 **심각도:** Medium

[ERR-M-02] 관리자 감사 trades — any 타입 사용 (Medium)

현상: order-proxy.controller.ts:164-176에서 as Record<string, any>, (t: any), (u: any) 등 any 타입이 5개소 사용됨. 타입 안전성 결여로 런타임 에러 가능, 프로퍼티 접근 시 undefined 크래시 위험 **위치:**

backend/services/api-gateway/src/proxy/order-proxy.controller.ts:164-176 **영향:** 예상치 못한 데이터 구조 변경 시 사일런트 실패 또는 크래시 **권장:** 응답 데이터에 대한 명시적 인터페이스 정의 후 타입 가드 적용 **심각도:** Medium

[ERR-L-01] 프론트엔드 catch 블록 — 에러 무시 패턴 반복 (Low)

현상: 다수 컴포넌트에서 `catch { // Error handled by query client }` 패턴으로 에러를 무시.
portfolio/page.tsx:99-101 , OrderForm.tsx:240-243 등에서 에러 객체를 참조하지 않아 디버깅 시 원인 추적 어려움 **위치:** frontend/src/app/(main)/portfolio/page.tsx:99 ,
frontend/src/components/trading/OrderForm.tsx:240 **영향:** 네트워크 에러, 서버 에러 등의 원인 파악 지연 **권장:** 최소한 `catch (e) { console.debug(e); }` 또는 에러 추적 서비스 연동 **심각도:** Low

D. XSS/CSP 방어 — 2건

[CSP-M-01] CSP script-src — unsafe-inline + strict-dynamic 공존 (Medium, 부분 수정)

현상: next.config.ts:9 에서 프로덕션 CSP `script-src` 가 'self' 'unsafe-inline' 'strict-dynamic' 으로 설정. CSP 3.0 사양에 따르면 'strict-dynamic' 이 있으면 'unsafe-inline' 은 CSP 3.0 지원 브라우저에서 무시되지만, CSP 2.0만 지원하는 브라우저에서는 'unsafe-inline' 이 적용되어 인라인 스크립트 실행 허용 **위치:** frontend/next.config.ts:9 **영향:** CSP 2.0 브라우저(일부 모바일 브라우저)에서 XSS 방어력 약화 **권장:** nonce 기반 CSP로 전환하여 'unsafe-inline' 완전 제거, 또는 CSP 2.0 폴백을 명시적으로 결정 **심각도:** Medium

[CSP-M-02] CSP connect-src — localhost 와일드카드 허용 (Medium)

현상: next.config.ts:38 의 CSP `connect-src` 에 `http://localhost:*` 가 포함. 프로덕션 환경에서 localhost 연결을 허용하면, 공격자가 XSS를 통해 코컬 서비스(Redis, DB 관리 도구 등)에 접근할 수 있는 경로를 제공 **위치:** frontend/next.config.ts:38 **영향:** XSS + 코컬 서비스 익스플로잇 체인 가능 **권장:** 프로덕션 빌드에서 `http://localhost:*` 를 제거하고 실제 API 도메인만 허용 **심각도:** Medium

E. 인가/권한 검증 — 2건

[AUTH-M-01] 호가창 엔드포인트 — 인증 불필요 노출 (Medium)

현상: order-proxy.controller.ts:212 의 GET `/api/orders/book/:symbol` 에 인증 가드가 없음. 호가창 데이터에 사용자들의 주문 가격/수량 정보가 포함되어 있어, 비인증 사용자가 시장 미시구조를 스크래핑할 수 있음 **위치:** backend/services/api-gateway/src/proxy/order-proxy.controller.ts:212 **영향:** 비인증 사용자가 호가 데이터를 자동 수집하여 시장 조작에 활용 가능 **권장:** 공개 API 여부를 명시적으로 결정하고, 비공개라면 JwtAuthGuard 추가. 공개로 결정 시 속도 제한 적용 **심각도:** Medium

[AUTH-L-01] 공개 포트폴리오 API — 보유 자산 상세 노출 범위 (Low)

현상: usePublicPortfolio(userId) 훅이 `/api/portfolio/public/${userId}` 로 다른 사용자의 보유 심볼, 수량, 평균가, 현재가, 수익률을 조회. 리더보드 프로필 모달에서 사용되지만, 정확한 포지션 정보 노출이 카피트레이딩 이외의 목적으로 악용될 수 있음 **위치:** frontend/src/hooks/usePortfolio.ts:152-171 **영향:** 특정 사용자의 정확한 포지션 크기가 노출되어 의도적 반대매매 가능 **권장:** 보유 비율(%)만 공개하고 절대 수량은 마스킹, 또는 본인 허용 설정 기반 공개 **심각도:** Low

F. 서비스 간 통신 — 2건

[SVC-M-01] JwtStrategy — SERVICE_HOST 대신 localhost 하드코딩 (Medium, 미수정)

현상: JwtStrategy에서 user-auth 서비스 URL을 `http://localhost:${port}` 로 생성. ProxyService는 SERVICE_HOST 환경변수를 사용하지만 JwtStrategy는 하드코딩. Docker/K8s 배포 시 통신 실패 **위치:**

`backend/services/api-gateway/src/auth/jwt.strategy.ts` 영향: 프로덕션 배포 시 인증 서비스 통신 불가 권장: `configService.getOrThrow<string>('SERVICE_HOST')` 사용 심각도: Medium

[SVC-L-01] 알림 DB 저장 — 주문 응답과 동기 (Low)

현상: `order-proxy.controller.ts:118-129` 에서 거래 알림을 DB에 저장하기 위해 `user-auth` 서비스로 `POST /notifications` 호출. `.catch()` 로 에러 처리하지만, `await` 없이 `.catch()` 만 체이닝되어 있어 fire-and-forget 으로 보이나, `sendTradeNotification` 자체가 `async` 메서드이므로 실행 순서 확인 필요 위치: `backend/services/api-gateway/src/proxy/order-proxy.controller.ts:118-129` 영향: 알림 서비스 지연 시 주문 응답 시간에 영향 가능 권장: 알림 저장을 명시적으로 `void sendTradeNotification()` 패턴으로 분리 심각도: Low

G. 프론트엔드 보안 — 2건

[FE-M-01] RichEditor 이미지 업로드 — 서버 실패 시 Base64 폴백 유지 (Medium)

현상: `RichEditor.tsx:144-147` 에서 서버 업로드 API 실패 시 `editor.chain().focus().setImage({ src: dataUrl })` 로 Base64 data URL 폴백. 서버 업로드 기능이 추가되었지만, 서버 장애나 네트워크 오류 시 여전히 대용량 Base64 데이터가 게시글 HTML에 포함될 수 있음 위치: `frontend/src/components/ui/RichEditor.tsx:144-147` 영향: 서버 불안정 시 DB에 대용량 Base64 데이터 저장, 게시글 조회 성능 저하 권장: 서버 업로드 실패 시 사용자에게 알림 후 이미지 삽입을 차단하거나, 재시도 로직 제공 심각도: Medium

[FE-L-01] window.confirm 사용 — 네이티브 다이얼로그 비일관성 (Low)

현상: 일부 채팅 컴포넌트에서 `window.confirm()` 사용. 커스텀 `ConfirmModal`이 프로젝트 전체에 적용되어 있으나 일부 위치에서 네이티브 confirm 사용으로 UI 일관성 저하 위치: `frontend/src/components/chat/ChatPanel.tsx` , `frontend/src/components/chat/RoomList.tsx` 영향: 커스텀 테마 미적용, 다크 모드에서 UI 불일치 권장: `ConfirmModal`로 통일 심각도: Low

H. 데이터 무결성 — 2건

[DI-M-01] 주문 폼 — displayCurrentPrice 기반 최대 수량 계산 편차 (Medium)

현상: `OrderForm.tsx:321` 에서 매수 100% 비율 클릭 시 `maxQty = portfolio.cashBalance / displayCurrentPrice` 로 계산. `displayCurrentPrice` 는 환율 변환이 적용된 표시 가격이므로, 실제 백엔드 USD 가격과 환율 변환 왕복 오차(반올림)로 인해 잔액 부족 에러가 발생할 수 있음 위치: `frontend/src/components/trading/OrderForm.tsx:320-323` 영향: 100% 매수 시 `factor = 0.99` 로 1% 여유를 두고 있으나, 환율 변환 오차가 1% 이상일 경우 여전히 실패 가능 권장: 최대 수량 계산을 백엔드 USD 가격 (`currentPrice`)과 백엔드 잔고 기준으로 수행하고, 표시만 변환 심각도: Medium

[DI-L-01] 커뮤니티 게시글 title — 프론트엔드 maxLength 미제한 (Low)

현상: 커뮤니티 게시글 작성 시 title 입력에 `maxLength` 속성이 없어 극단적으로 긴 제목 입력 가능. 백엔드 DTO에서 `@MaxLength()` 적용 여부 확인 필요 위치: `frontend/src/app/(main)/community/new/page.tsx` 영향: DB 저장 실패 또는 게시글 목록 UI 깨짐 권장: 프론트엔드 title input에 `maxLength={200}` 제한 추가 심각도: Low

종합 평가

20차 시스템 감사에서 총 17건의 이슈를 발견하였습니다. 19차 대비 관리자 엔드포인트 권한 검증(AUTH-M-01/02), 주문 수정 DTO 타입 적용(VAL-M-01) 등 3건이 수정되었고, CSP 설정이 부분 개선되었습니다.

핵심 미수정 이슈:

1. **JWT sessionStorage 저장** (SEC-C-01, Critical) — 프로덕션 출시 전 반드시 HttpOnly 쿠키로 전환 필요
2. **JwtStrategy localhost 하드코딩** (SVC-M-01) — 배포 환경에서 인증 서비스 통신 실패 유발

새로 발견된 주요 이슈:

1. **CSP connect-src localhost 와일드카드** (CSP-M-02) — XSS + 로컬 서비스 체인 공격 경로
2. **RichEditor Base64 풀백** (FE-M-01) — 서버 업로드 실패 시 여전히 대용량 데이터 저장

SEC-C-01(JWT sessionStorage) + CSP-M-01(unsafe-inline) + CSP-M-02(localhost 허용)의 조합은 XSS 공격 시 토큰 탈취 + 로컬 서비스 접근이라는 연쇄 공격 벡터를 형성할 수 있어, 이 세 가지의 우선 수정을 강력히 권장합니다.

본 보고서는 자동 생성된 감사 결과이며, 수정 사항은 포함되지 않습니다.